

EXPERIMENT 18

By Trishil Chauhan

Aim

To write a menu-driven C program using **switch-case** for digit-based operations on a positive integer. A separate **void user-defined function** is used for every menu option. Each function reads the required input, processes it, and displays the result.

Required Operations

1. Check whether the number is a Strong number.
2. Check whether the number is an Armstrong number.
3. Check whether the number is a Harshad number.
4. Check whether the number is a Duck number.

Definitions

Strong number: A number equal to the sum of factorials of its digits. Example: $145 = 1! + 4! + 5!$.

Armstrong number: A number equal to the sum of each digit raised to the number of digits. Example: $153 = 1^3 + 5^3 + 3^3$.

Harshad number: A positive integer divisible by the sum of its digits. Example: 18 is divisible by $1 + 8 = 9$.

Duck number: A positive integer containing at least one zero after its first digit. Example: 1023.

Algorithm

Main algorithm

1. Start.
2. Display four menu options.
3. Read the user's choice.
4. Use switch-case to call strongNumber(), armstrongNumber(), harshadNumber(), or duckNumber().
5. Each void function reads a positive integer, performs its own calculation, and prints the result.
6. If the menu choice is not 1-4, print "Invalid choice".
7. Stop.

Strong number: Extract every digit, find its factorial, add the factorials, and compare the sum with the original number.

Armstrong number: Count the digits. For each digit, calculate digit raised to that count, add the values, and compare the sum with the original number.

Harshad number: Add all digits. If the original number is exactly divisible by the digit sum, it is a Harshad number.

Duck number: Read the positive integer as a string and check whether any character after the first digit is 0.

Test Case Table

Test	Choice	Input	Expected Output
1	1	145	Strong number
2	1	123	Not a Strong number

Test	Choice	Input	Expected Output
3	2	153	Armstrong number
4	2	123	Not an Armstrong number
5	3	18	Harshad number
6	3	19	Not a Harshad number
7	4	1023	Duck number
8	4	1234	Not a Duck number
9	5	-	Invalid choice

C Program

```
#include <stdio.h>
#include <string.h>

void strongNumber(void);
void armstrongNumber(void);
void harshadNumber(void);
void duckNumber(void);

int powerInt(int base, int exp)
{
    int result = 1;
    while (exp-- > 0)
        result *= base;
    return result;
}

int main(void)
{
    int choice;

    printf("1. Check Strong number\n");
    printf("2. Check Armstrong number\n");
    printf("3. Check Harshad number\n");
    printf("4. Check Duck number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: strongNumber(); break;
        case 2: armstrongNumber(); break;
        case 3: harshadNumber(); break;
        case 4: duckNumber(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}

void strongNumber(void)
{
    int n, original, temp, digit, i, fact, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    original = n;
    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        fact = 1;
        for (i = 1; i <= digit; i++)
            fact *= i;
        sum += fact;
        temp /= 10;
    }

    if (sum == original)
        printf("Strong number\n");
    else
        printf("Not a Strong number\n");
}

void armstrongNumber(void)
{
    int n, original, temp, digit, digits = 0, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    original = n;
    temp = n;
    while (temp > 0) {
```

```

        digits++;
        temp /= 10;
    }

    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        sum += powerInt(digit, digits);
        temp /= 10;
    }

    if (sum == original)
        printf("Armstrong number\n");
    else
        printf("Not an Armstrong number\n");
}

void harshadNumber(void)
{
    int n, original, temp, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    original = n;
    temp = n;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }

    if (original % sum == 0)
        printf("Harshad number\n");
    else
        printf("Not a Harshad number\n");
}

void duckNumber(void)
{
    char n[50];
    int i, found = 0;
    printf("Enter a positive integer: ");
    scanf("%49s", n);

    if (n[0] == '-' || strcmp(n, "0") == 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    for (i = 1; n[i] != '\0'; i++) {
        if (n[i] == '0') {
            found = 1;
            break;
        }
    }

    if (found)
        printf("Duck number\n");
    else
        printf("Not a Duck number\n");
}

```

Compilation and Execution

Save as: experiment18.c

Compile: gcc experiment18.c -o experiment18

Execute: ./experiment18

Sample Executions

Strong number test

Enter your choice: 1

Enter a positive integer: 145

Strong number

Armstrong number test

Enter your choice: 2

Enter a positive integer: 153

Armstrong number

Harshad number test

Enter your choice: 3

Enter a positive integer: 18

Harshad number

Duck number test

Enter your choice: 4

Enter a positive integer: 1023

Duck number

Result

The menu-driven C program was prepared using switch-case and separate void user-defined functions for all four menu options. The program correctly checks Strong, Armstrong, Harshad, and Duck numbers for the prepared test cases.